ASSIGNMENT 2

Student Course Registration System

NAME: Ishmal Sheikh & Aamna Basharat

ROLL NO: 24P-3054 & 24P-3121

SECTION: BSE-4B

COURSE: SOFTWARE QUALITY ENGINEERING

GITHUB: https://github.com/AamnaBasharat7/SQE-Assignment-2

Part A: Black-Box Testing

A1. Equivalence Class Partitioning

Input data is divided into valid and invalid classes, and one representative value is tested from each class.

Table A1: Equivalence Class Partitioning

Test ID	Requirement ID	Input Area	Equivalence Class	Class Type	Representative Input	Expected Result
ECP-01	FR-01, FR-02	Current credits	0–18 inclusive	Valid	10	Accepted for credit calculation
ECP-02	FR-02	Current credits	Below 0	Invalid	-1	ValueError: Invalid current credits
ECP-03	FR-02	Current credits	Above 18	Invalid	19	ValueError: Invalid current credits
ECP-04	FR-01, FR-02	Course credits	1–4 inclusive	Valid	3	Accepted for credit calculation
ECP-05	FR-02	Course credits	Below 1	Invalid	0	ValueError: Invalid course credits
ECP-06	FR-02	Course credits	Above 4	Invalid	5	ValueError: Invalid course credits
ECP-07	FR-04, FR-09	Outstanding dues	Exactly zero	Valid	Rs. 0	No dues block
ECP-08	FR-04, FR-09	Outstanding dues	Positive amount	Valid input, registration blocked	Rs. 1	DUES_BLOCK
ECP-09	FR-15	Outstanding dues	Negative amount	Invalid	Rs. -1	ValueError: Invalid dues
ECP-10	FR-03, FR-06	Prerequisites	No prerequisites required	Valid	Empty prerequisite set	Prerequisites satisfied
ECP-11	FR-03, FR-06	Prerequisites	All prerequisites completed	Valid	All required courses completed	Prerequisites satisfied
ECP-12	FR-03, FR-06	Prerequisites	At least one missing	Invalid for registration	One required course missing	PREREQUISITES_MISSING
ECP-13	FR-08, FR-09	Availability	At least one seat available	Valid	Capacity 30, enrolled 29	REGISTERED if other checks pass
ECP-14	FR-08, FR-09	Availability	Course exactly full	Valid input, waitlisted	Capacity 30, enrolled 30	WAITLISTED if other checks pass
ECP-15	FR-15	Availability	Invalid capacity or enrollment	Invalid	Capacity 0, or enrollment below 0 or above capacity	ValueError: Invalid capacity or enrollment
ECP-16	FR-05, FR-09	Duplicate registration	Course already registered	Invalid for new registration	Course code already in registered courses	ALREADY_REGISTERED

A2. Boundary Value Analysis

Values at, just below and just above each limit are tested, since errors tend to occur at the edges of valid ranges.

Table A2: Boundary Value Analysis

Test ID	Requirement ID	Parameter	Boundary	Input	Expected Result
BVA-01	FR-02	Current credits	Just below minimum	-1	ValueError: Invalid current credits
BVA-02	FR-02	Current credits	Minimum	0	Accepted
BVA-03	FR-01	Current credits	Just above minimum	1	Accepted
BVA-04	FR-01	Combined credits	Exact maximum	Current 17 + course 1 = 18	True
BVA-05	FR-01	Combined credits	Just above maximum	Current 18 + course 1 = 19	False
BVA-06	FR-02	Current credits	Maximum	18	Accepted as current credits
BVA-07	FR-02	Current credits	Just above maximum	19	ValueError: Invalid current credits
BVA-08	FR-02	Course credits	Just below minimum	0	ValueError: Invalid course credits
BVA-09	FR-02	Course credits	Minimum	1	Accepted
BVA-10	FR-02	Course credits	Maximum	4	Accepted
BVA-11	FR-02	Course credits	Just above maximum	5	ValueError: Invalid course credits
BVA-12	FR-01	Combined credits	Exact maximum	Current 14 + course 4 = 18	True
BVA-13	FR-01	Combined credits	Just above maximum	Current 15 + course 4 = 19	False
BVA-14	FR-08, FR-15	Course capacity	One seat remaining	Capacity 1, enrolled 0	REGISTERED
BVA-15	FR-08	Course capacity	Exactly full	Capacity 1, enrolled 1	WAITLISTED
BVA-16	FR-15	Enrollment	Below minimum	Capacity 1, enrolled -1	ValueError: Invalid capacity or enrollment
BVA-17	FR-15	Enrollment	Above capacity	Capacity 1, enrolled 2	ValueError: Invalid capacity or enrollment
BVA-18	FR-15	Capacity	Below minimum	Capacity 0, enrolled 0	ValueError: Invalid capacity or enrollment
BVA-19	FR-04, FR-09	Outstanding dues	Zero boundary	Rs. 0	No dues block
BVA-20	FR-04, FR-09	Outstanding dues	Smallest positive amount	Rs. 1	DUES_BLOCK

A3. Decision Table Testing

The decision table lists combinations of conditions and the expected registration result. When more than one condition fails, the first check in this priority order decides the outcome: outstanding dues, duplicate registration, missing prerequisites, credit-hour limit, seat availability.

Table A3: Registration Decision Table

Rule	Dues > 0?	Already registered?	Prerequisites missing?	Credit limit exceeded?	Seat available?	Expected outcome
R1	Yes	Yes	Yes	Yes	No	DUES_BLOCK
R2	No	Yes	Yes	Yes	No	ALREADY_REGISTERED
R3	No	No	Yes	Yes	No	PREREQUISITES_MISSING
R4	No	No	No	Yes	No	CREDIT_LIMIT
R5	No	No	No	No	No	WAITLISTED
R6	No	No	No	No	Yes	REGISTERED
R7	Yes	No	No	No	Yes	DUES_BLOCK
R8	No	Yes	No	No	Yes	ALREADY_REGISTERED

Table A3.1: Meaning of the decision table rules

Rule	Explanation
R1	Dues block registration even when other conditions also fail.
R2	Duplicate registration is checked before prerequisites and credit eligibility.
R3	Missing prerequisites block registration before credit and seat checks.
R4	Exceeding the credit limit blocks registration before the seat check.
R5	An eligible student is waitlisted when the course is full.
R6	An eligible student is registered when a seat is available.
R7	Positive dues block registration even when all later checks pass.
R8	Duplicate registration blocks registration even when all later checks pass.

The table covers all six distinct outcomes: DUES_BLOCK, ALREADY_REGISTERED, PREREQUISITES_MISSING, CREDIT_LIMIT, WAITLISTED and REGISTERED. R1–R4, R7 and R8 also show the priority order.

A4. Pytest Implementation and Execution

Table A4.1: Initial Black-Box Test Execution Summary

Description	Actual Result
Test file	tests/test_blackbox.py
Testing framework	pytest
Total tests executed	36
Passed tests	26
Failed tests	10
Execution time	2.31 seconds
Result	10 failures identified in original implementation

The black-box suite was executed against the original implementation. Of 36 tests, 26 passed and 10 failed. The failures indicate possible defects in dues validation, prerequisite checking, credit-hour limits and course capacity handling.

Table A4.2: Identified defects

Defect	Requirement	Description	Failed test evidence
DEF-01	FR-04	Positive outstanding dues did not block registration correctly.	ECP02, DT02, STATE04
DEF-02	FR-03	Registration accepted students with incomplete prerequisites.	ECP06
DEF-03	FR-08, FR-09, FR-12	Full courses did not correctly place students on the waiting list.	ECP10, DT06, STATE02
DEF-04	FR-01	A total of exactly 18 credits was incorrectly rejected.	BVA06, BVA09
DEF-05	FR-13	Withdrawal did not decrement course enrollment.	STATE05

Black-box testing identified five functional defects. They were documented as GitHub Issues #1 to #5, corrected, and verified through regression testing. The final combined run passed all 46 tests.

Part B: White-Box Testing

B1. CFG and Cyclomatic Complexity

B1.1 Control Flow Graph

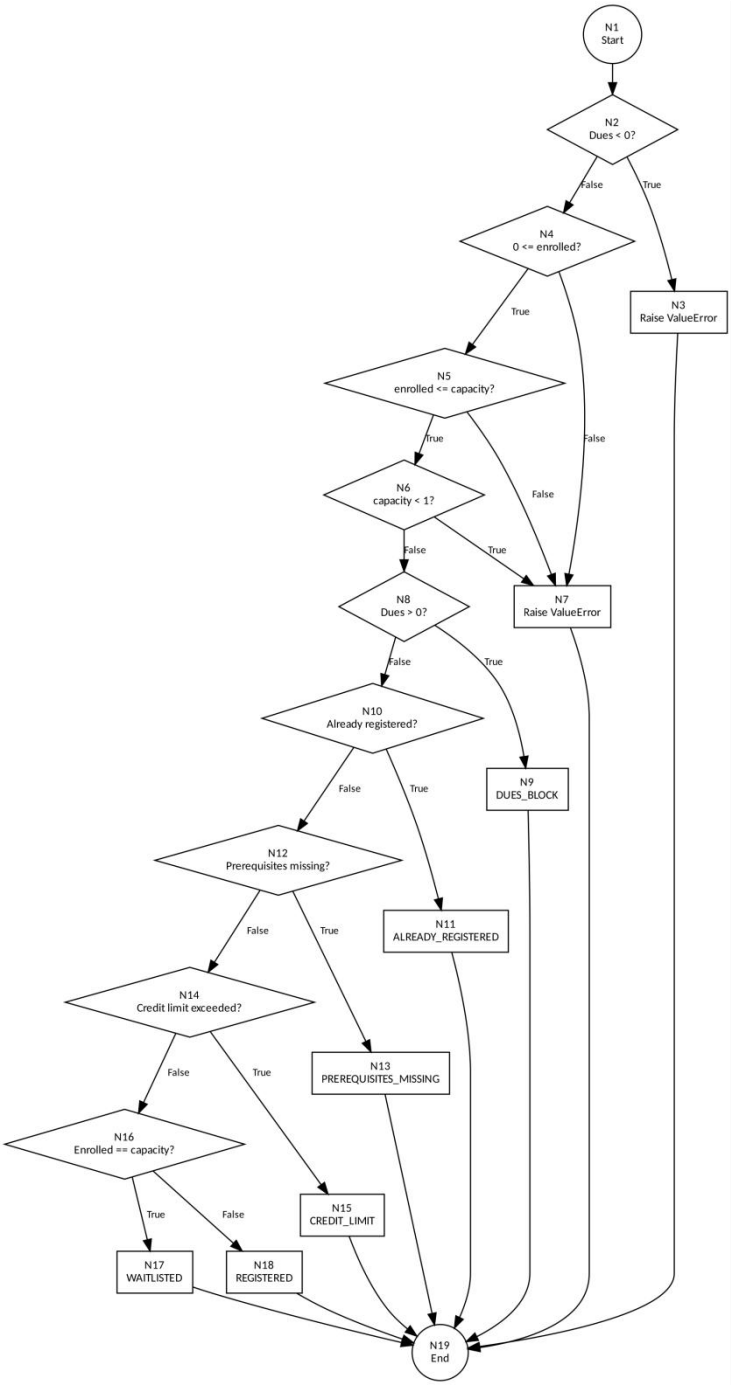


Figure B1.1: Control Flow Graph of registration_decision()

B1.2 CFG Nodes

Node	Type	Description
N1	Start	Start of registration decision
N2	Decision	Outstanding dues less than zero?
N3	Return	Raise invalid dues error
N4	Decision	Enrollment greater than or equal to zero?
N5	Decision	Enrollment less than or equal to capacity?
N6	Decision	Capacity less than one?
N7	Return	Raise invalid capacity error
N8	Decision	Outstanding dues greater than zero?
N9	Return	DUES_BLOCK
N10	Decision	Course already registered?
N11	Return	ALREADY_REGISTERED
N12	Decision	Prerequisites missing?
N13	Return	PREREQUISITES_MISSING
N14	Decision	Credit limit exceeded?
N15	Return	CREDIT_LIMIT
N16	Decision	Enrollment equals capacity?
N17	Return	WAITLISTED
N18	Return	REGISTERED
N19	End	End of function

B1.3 Cyclomatic Complexity

V(G) = E − N + 2P, where E is the number of edges, N the number of nodes and P the number of connected components.

Element	Value
Nodes (N)	19
Edges (E)	27
Connected components (P)	1

Calculation: V(G) = 27 − 19 + 2(1) = **10**

Cross-check: the graph has nine decision nodes (N2, N4, N5, N6, N8, N10, N12, N14, N16), so V(G) = 9 + 1 = 10.

The expanded short-circuit convention is used: the three atomic conditions in the capacity-validation expression (N4, N5, N6) are counted as separate decisions. If that expression were counted as one source-level decision, V(G) would be 8.

B1.4 Independent Basis Paths

Table B1.4: Basis path set

Path	Execution condition	Expected result	Representative test
P1	Outstanding dues are negative	ValueError	WB01
P2	Dues valid, enrollment is negative	ValueError	WB-C02
P3	Enrollment is greater than capacity	ValueError	WB-C03
P4	Capacity is zero	ValueError	WB-C04
P5	Outstanding dues are positive	DUES_BLOCK	WB02
P6	Course is already registered	ALREADY_REGISTERED	WB03
P7	Required prerequisites are missing	PREREQUISITES_MISSING	WB04
P8	Credit limit is exceeded	CREDIT_LIMIT	WB05
P9	All earlier checks pass and course enrollment equals capacity	WAITLISTED	ECP-14 / DT06
P10	All checks pass	REGISTERED	WB06

Important finding: in the original implementation, P9 was unreachable. The earlier capacity validation rejected enrollment greater than capacity, so the waitlist condition could never be True. After the defect fix, P9 is a valid, reachable path: WAITLISTED is returned when all earlier checks pass and enrollment equals capacity.

B2. Statement and Branch Coverage

B2.1 Initial, Pre-Fix and Post-Defect-Fix Coverage

Coverage was measured with pytest-cov (branch coverage enabled). The initial and pre-fix columns are kept as historical evidence; the post-defect-fix column is the final run after all five defects were fixed.

Table B2.1: Coverage before and after white-box additions and defect fixes

Measure	Initial (black-box)	Pre-fix (combined)	Post-defect-fix (final)
Total tests	36	46	46
Passed tests	26	36	46
Failed tests	10	10	0
Total statements	53	53	52
Covered statements	49	50	52
Missing statements	4	3	0
Statement coverage	92.45%	94.34%	100%
Total branches	26	26	24
Covered branches	21	22	24
Missing branches	5	4	0
Partially covered branches	3	2	0
Branch coverage	80.77%	84.62%	100%

Measure	Initial (black-box)	Pre-fix (combined)	Post-defect-fix (final)
Combined coverage	88.61%	91.14%	100%

Calculations:

statement coverage = covered statements ÷ total statements (49/53 = 92.45%; 50/53 = 94.34%; post-fix 52/52 = 100%). Branch coverage = covered branches ÷ total branches (21/26 = 80.77%; 22/26 = 84.62%; post-fix 24/24 = 100%).

Combined coverage = (covered statements + covered branches) ÷ (total statements + total branches) (70/79 = 88.61%; 72/79 = 91.14%; post-fix 76/76 = 100%).

The statement and branch totals in the post-fix column differ from the earlier columns because that run measures the corrected implementation.

B2.2 Uncovered Locations

The initial report showed uncovered lines 51, 59, 70 and 71 of registration.py. After the white-box tests, line 51 was covered and these remained in the pre-fix implementation:

Table B2.2: Uncovered locations (pre-fix)

Line	Code location	Reason
59	`return "WAITLISTED" in registration_decision()`	Unreachable: the earlier capacity validation rejects enrollment above capacity.
70	`elif decision == "WAITLISTED" in register_course()`	No test can reach this branch while WAITLISTED is unreachable.
71	`course.waitlist.append(student.student_id)`	The waitlist append is never executed.

Added structural tests: WB-C01 to WB-C04 and WB01 to WB06 (Table B4.1).

The ten white-box tests raised statement coverage by 1.89 percentage points and branch coverage by 3.85 points. In the pre-fix implementation, the remaining gaps all belonged to the waitlist functionality, which could not be executed without correcting the implementation.

Post-defect-fix result: after all five defects were fixed, the final run covers 52/52 statements and 24/24 branches, with no partially covered branches. The waitlist logic that was previously unreachable is now executed, because WAITLISTED is returned when enrollment equals capacity, so no uncovered locations remain.

B3. Condition Coverage and MC/DC

B3.1 Compound Decision and Atomic Conditions

```
if not 0 <= course.enrolled <= course.capacity or course.capacity < 1:
    raise ValueError("Invalid capacity or enrollment")
```

This decision contains three atomic conditions:

Condition	Meaning
A	`0 <= course.enrolled`
B	`course.enrolled <= course.capacity`

Condition	Meaning
C	`course.capacity < 1`

The overall decision is D = ¬(A ∧ B) ∨ C. If D is True the program raises ValueError; if D is False it continues with the registration checks.

B3.2 Truth Table with Short-Circuit Evaluation

Python skips conditions once the result is known. N/E means Not Evaluated.

Test ID	A	B	C	Decision D	Expected result
WB-C01	T	T	F	F	REGISTERED
WB-C02	F	N/E	N/E	T	ValueError
WB-C03	T	F	N/E	T	ValueError
WB-C04	T	T	T	T	ValueError

WB-C01: enrollment is within capacity and capacity is valid, so all three conditions are evaluated.

WB-C02: enrollment is negative. A is False, so B and C are skipped.

WB-C03: enrollment is greater than capacity. A is True and B is False, so C is skipped.

WB-C04: enrollment and capacity are both zero. A and B are True, so C is evaluated and is True.

B3.3 MC/DC Independent Pairs

MC/DC requires each atomic condition to independently change the decision outcome. In each pair only the listed condition changes; the other conditions keep the same logical values.

Condition	Test pair	Change	Other conditions held at	Decision change
A	WB-C01, WB-C02	A: T → F	B = T, C = F	F → T
B	WB-C01, WB-C03	B: T → F	A = T, C = F	F → T
C	WB-C01, WB-C04	C: F → T	A = T, B = T	F → T

In WB-C02 and WB-C03, Python does not evaluate B and C (or just C), but their logical values are unchanged from WB-C01, so the short-circuit does not affect the pair. All three conditions therefore have an MC/DC pair using four tests (n + 1 for n = 3). Each condition also takes both True and False across the tests, which gives full condition coverage.

Condition coverage and MC/DC were calculated manually, because pytest-cov measures only statement and branch coverage.

B4. White-Box Pytest Tests

Ten tests were added in tests/test_whitebox.py. Each one targets a specific node, condition or path of registration_decision().

Table B4.1: White-box tests and results

Test ID	Test purpose	Targeted branch, condition or path	Result
WB-C01	Valid capacity conditions	A = T, B = T, C = F; N4–N6 pass and execution	PASS

Test ID	Test purpose	Targeted branch, condition or path	Result
		continues	
WB-C02	Negative enrollment	Condition A False at N4 → N7 (P2)	PASS
WB-C03	Enrollment above capacity	Condition B False at N5 → N7 (P3)	PASS
WB-C04	Zero capacity	Condition C True at N6 → N7 (P4)	PASS
WB01	Negative dues validation	N2 True → N3 (P1)	PASS
WB02	Dues-block decision for any positive dues	N8 True (dues > 0) → N9 (P5)	PASS
WB03	Duplicate registration	N10 True → N11 (P6)	PASS
WB04	Missing prerequisite	N12 True → N13 (P7)	PASS
WB05	Credit limit exceeded	N14 True → N15 (P8)	PASS
WB06	Successful registration	All checks pass → N18 (P10)	PASS

Table B4.2: White-box execution summary

Metric	Result
Total white-box tests	10
Passed	10
Failed	0
Execution time	0.39 seconds

Part C: GitHub Collaboration and Defect Management

C1. Repository Setup

A GitHub repository named SQE-Assignment-2 was created to hold the source code, requirements, test cases, testing evidence and documentation. Repository URL: https://github.com/AamnaBasharat7/SQE-Assignment-2

Table C1: Repository contents

File / Folder	Purpose
registration.py	Student course registration system implementation
REQUIREMENTS.md	Software requirements specification
requirements.txt	Project dependencies
README.md	Project overview and setup instructions
tests/test_blackbox.py	Black-box test cases
tests/test_whitebox.py	White-box test cases
docs/blackbox_test_design.xlsx	Black-box test design evidence
docs/decision_table.xlsx	Decision table test design
docs/cfg.png	Control Flow Graph
docs/testing_report.pdf	Final testing report
.github/workflows/tests.yml	GitHub Actions automated testing workflow

The .gitignore file excludes virtual environments, Python cache files and temporary coverage data from version control.

C2. Branches and Commits

Feature branches were used to organize the testing and development work.

Table C2: Repository branches

Branch	Purpose
main	Main project branch
blackbox-testing	Black-box testing and workflow-related work
whitebox-testing	White-box testing work
defect-fixes	Corrections to identified defects and final coverage evidence
documentation-report	Testing documentation and report files

Descriptive commits recorded the purpose of each change: initial project setup, addition of the GitHub Actions workflow, correction of registration defects, updating of final coverage evidence, and addition of the testing spreadsheets, CFG diagram and report documentation.

C3. GitHub Issues and Defect Management

Each defect found during testing was recorded as a GitHub Issue with the affected requirement, a description and the failed test evidence.

Table C3: GitHub Issues

Issue	Requirement	Defect	Failed test evidence	Status
#1	FR-04	Positive outstanding dues did not block registration	ECP02, DT02, STATE04	Closed
#2	FR-03	Incomplete prerequisites were incorrectly accepted	ECP06	Closed
#3	FR-08, FR-09, FR-12	Full courses did not correctly place students on the waiting list	ECP10, DT06, STATE02	Closed
#4	FR-01	Exactly 18 credits were incorrectly rejected	BVA06, BVA09	Closed
#5	FR-13	Withdrawal did not decrement course enrollment	STATE05	Closed

Table C3.1: Defect resolution

Issue	Correction
#1	Changed the outstanding-dues condition so that any positive balance blocks registration.
#2	Corrected prerequisite validation to verify that all required courses are completed.
#3	Corrected the full-course condition so that enrollment equal to capacity results in WAITLISTED.
#4	Corrected the credit-limit condition to allow a total of exactly 18 credits.
#5	Decremented course enrollment after a successful withdrawal.

All five issues were closed after the corrections were made.

C4. Pull Requests and Code Review

Pull requests were used to propose and review changes before merging them into the main branch.

Table C4: Pull requests

PR	Title	Purpose	Status
#6	Add GitHub Actions workflow for automated testing	Add automated testing workflow	Merged
#7	Fix registration defects and update final coverage evidence	Correct defects and update test evidence	Merged
#8	Add testing documentation and final report	Add testing spreadsheets, CFG and report	Open at last check

Pull Request #7 was reviewed and approved by teammate iishmal before it was merged. Pull Request #8 adds the black-box test design spreadsheet, the decision table spreadsheet, the CFG diagram and the final testing report PDF.

C5. GitHub Actions

GitHub Actions runs the tests automatically when changes are pushed or pull requests are created. The workflow file is .github/workflows/tests.yml.

Table C5.1: Workflow steps

Step	Activity
1	Checks out the repository
2	Sets up the Python environment
3	Installs project dependencies
4	Executes the pytest test suite
5	Measures statement and branch coverage
6	Generates coverage output
7	Uploads test and coverage reports as workflow artifacts

Table C5.2: Workflow results

Metric	Result
Total tests	46
Passed	46
Failed	0
Statement coverage	100%
Branch coverage	100%
Workflow result	Successful

The workflow uploaded the pytest-and-coverage-reports artifact. Non-blocking platform deprecation warnings were displayed, but they did not prevent the workflow from completing successfully.

Part D: Final Summary

D1. Testing Techniques Summary

Table D1: Techniques applied

Technique	Application	Result
Equivalence Class Partitioning	Divided inputs into valid and invalid classes	16 test cases
Boundary Value Analysis	Tested values around important limits	20 test cases
Decision Table Testing	Tested combinations of registration conditions	8 decision rules
Control Flow Graph	Represented program execution flow	CFG created
Cyclomatic Complexity	Calculated independent path complexity	10 (expanded atomic-condition CFG)
Independent Path Testing	Tested separate execution paths	10 paths
MC/DC	Verified independent effects of atomic conditions	4 capacity tests

D2. Coverage Summary

Table D2: Coverage before and after

Metric	Initial black-box	Final
Statement coverage	92.45%	100%
Branch coverage	80.77%	100%
Total tests	36	46
Passed tests	26	46
Failed tests	10	0

All measured executable statements and branches in the registration module were covered in the final run.

D3. Defect Summary

Table D3: Defects and fix status

Defect	Requirement	Resolution	Issue status
Positive dues did not block registration	FR-04	Corrected dues condition	Closed
Missing prerequisites were accepted	FR-03	Corrected prerequisite validation	Closed
Full courses were not waitlisted	FR-08, FR-09, FR-12	Corrected capacity decision	Closed
Exactly 18 credits were rejected	FR-01	Corrected credit-limit comparison	Closed
Withdrawal did not decrement enrollment	FR-13	Added enrollment decrement	Closed

D4. GitHub Collaboration Summary

Table D4: GitHub references

Item	Reference

Item	Reference
Repository	https://github.com/AamnaBasharat7/SQE-Assignment-2
Branches	main, blackbox-testing, whitebox-testing, defect-fixes, documentation-report
Issues	#1 to #5, one for each defect (all closed)
Pull Request #6	GitHub Actions workflow (merged)
Pull Request #7	Defect corrections and coverage evidence (merged, reviewed by iishmal)
Pull Request #8	Testing documentation and final report (open at last check)
GitHub Actions	Automated pytest and coverage run on push and pull request